

Folder Structure

```
📁 Pollinator-Habitat
├── 📁 backend - Node.js/Express REST API
│   ├── 📁 JsonDataBackups - JSON exports of game content for reference and re-seeding
│   │   ├── 📁 data
│   │   │   ├── 📄 Routes.json - Secondary copy of routes
│   │   │   ├── 📄 FactNodes.json - Backup of all pollinator facts
│   │   │   ├── 📄 RouteNodes.json - Backup of all route narrative nodes
│   │   │   └── 📄 Routes.json - Backup of all routes
│   │   └── 📁 prisma - Database schema, migrations, and seeds
│   │       ├── 📁 migrations - MySQL migration history
│   │       │   ├── 📄 20260224061121_init
│   │       │   │   └── 📄 migration.sql - Initial schema
│   │       │   ├── 📄 20260228120000_add_player_session_num_children_adults
│   │       │   │   └── 📄 migration.sql - Adds party size tracking
│   │       │   ├── 📄 20260316200703_add_survey_fields
│   │       │   │   └── 📄 migration.sql - Adds survey data fields
│   │       │   ├── 📄 20260419000000_drop_unused_total_routes_completed
│   │       │   │   └── 📄 migration.sql - Schema cleanup
│   │       │   └── 📄 migration_lock.toml - Locks migration engine to prevent conflicts
│   │       ├── 📁 postgresql - PostgreSQL variant of schema and migrations
│   │       │   ├── 📁 migrations - PostgreSQL migration history (mirrors MySQL)
│   │       │   │   ├── 📄 20260224061121_init
│   │       │   │   │   └── 📄 migration.sql
│   │       │   │   ├── 📄 20260228120000_add_player_session_num_children_adults
│   │       │   │   │   └── 📄 migration.sql
│   │       │   │   ├── 📄 20260316200703_add_survey_fields
│   │       │   │   │   └── 📄 migration.sql
│   │       │   │   ├── 📄 20260419000000_drop_unused_total_routes_completed
│   │       │   │   │   └── 📄 migration.sql
│   │       │   │   └── 📄 migration_lock.toml
│   │       │   └── 📄 schema.prisma - PostgreSQL-specific Prisma data model
│   │       ├── 📄 prisma-uml.png - ER diagram of all 7 database models
│   │       └── 📄 schema.prisma - MySQL Prisma data model (Session, PlayerSession, RouteCycle,
│   │           Route, RouteNode, RouteNodeOnRoute, FactNode)
│   ├── 📄 seed.js - Populates database with routes and facts for MySQL
│   └── 📄 seed.pg.js - Populates database with routes and facts for PostgreSQL
├── 📁 src
│   ├── 📁 Back-end_types
│   │   └── 📄 route.dto.ts - Route data transfer object (typed API response shape)
│   ├── 📁 api
│   │   ├── 📁 Utilities
│   │   │   ├── 📄 api.utilities.ts - JWT validation and auth helper functions
│   │   │   └── 📄 reshape.Utilites.ts - Response formatting and data reshaping utilities
│   │   ├── 📁 interfaces
│   │   │   └── 📄 api.interfaces.ts - Request and response type definitions
│   │   └── 📄 api.ts - All REST endpoint handlers (routes, surveys, stats, auth)
│   ├── 📁 middleware
│   │   └── 📄 middleware.ts - JWT auth, CORS, and rate limiting middleware
│   └── 📁 services
```

```

| | | | | db
| | | | | DatabaseOperationsServices
| | | | | Create
| | | | | getOrCreatePlayerSession.service.ts - Creates or retrieves a player
session record
| | | | | Read
| | | | | GetpollinatorNames.ts - Fetches all pollinator names from DB
| | | | | LoadRouteDTO.service.ts - Assembles a complete route with nodes and
facts
| | | | | getActiveRouteForPlayerSession.service.ts - Gets the current active
route for a player
| | | | | getAvgRoutesPerSessionByDateRange.ts - Analytics: avg routes per
session filtered by date
| | | | | getAvgRoutesPerSessionBySession.ts - Analytics: avg routes per
session
| | | | | getAvgRoutesPerUserByDateRange.ts - Analytics: avg routes per user
filtered by date
| | | | | getAvgRoutesPerUserBySession.ts - Analytics: avg routes per user
| | | | | getChildrenandAdultsbySession.ts - Gets child and adult counts
grouped by session
| | | | | getReplayRatioByDateRange.ts - Analytics: replay ratio filtered by
date
| | | | | getReplayRatioBySession.ts - Analytics: replay ratio per session
| | | | | getSessionsandPlayeridsbyChildSize.ts - Filters sessions by number of
children
| | | | | getTotalConnectedUsers.ts - Analytics: total unique users ever
| | | | | getTotalConnectedUsersByDateRange.ts - Analytics: unique users
filtered by date
| | | | | getTotalReplays.ts - Analytics: total route replays ever
| | | | | getTotalReplaysByDateRange.ts - Analytics: total replays filtered by
date
| | | | | getTotalRoutesCompleted.ts - Analytics: total routes completed ever
| | | | | getTotalRoutesCompletedByDateRange.ts - Analytics: routes completed
filtered by date
| | | | | getUsersCompletedMoreThanOneRoute.ts - Analytics: users who finished
more than one route
| | | | | getUsersCompletedMoreThanOneRouteByDateRange.ts - Analytics: same,
filtered by date
| | | | | getUsersNeverStartedRoute.ts - Analytics: users who connected but
never started
| | | | | getUsersNeverStartedRouteByDateRange.ts - Analytics: same, filtered
by date
| | | | | getUsersStartedButNotFinishedByDateRange.ts - Analytics: users who
started but did not finish, filtered by date
| | | | | getUsersStartedMoreThanOneRoute.ts - Analytics: users who started
more than one route
| | | | | getUsersStartedMoreThanOneRouteByDateRange.ts - Analytics: same,
filtered by date
| | | | | getchildadultdatabystartandend.ts - Gets child/adult data between two
dates
| | | | | getchildnumandadultdatetoforever.ts - Gets child/adult data from a
start date to present

```

```
| | | | | } 📁 getChildNumAndAdultToEndDate.ts - Gets child/adult data up to an end date
| | | | | } 📁 getSessionandPlayeridbyAdultsize.ts - Filters sessions by adult party size
| | | | | } 📁 getSessionidsandplayeridsbyfamilysize.ts - Filters sessions by total family size
| | | | | } 📁 getUserswhostartedbutdidnotfinishanyroute.ts - Analytics: users who never finished any route
| | | | | } 📁 hassession.service.ts - Checks whether a session ID exists in the DB
| | | | | } 📁 Transactions
| | | | | } 📁 getNextRouteForPlayerSession.service.ts - Core cycle algorithm; assigns next route avoiding repeats
| | | | | } 📁 Update
| | | | | } 📁 addChildrenadults.ts - Records party size (children + adults) at survey time
| | | | | } 📁 prisma.ts - Prisma client initialization and export
| | | | | } 📁 gameServiceUtils
| | | | | } 📁 CheckSession.service.ts - Validates that a session code exists and is active
| | | | | } 📁 PlayeridTypeCheck.ts - Runtime type guard for player ID format
| | | | | } 📁 game.service.ts - Orchestrates player sessions and route assignment
| | | | | } 📁 route.service.ts - Route management and deduplication logic
| | | | | } 📁 app.ts - Express app setup, middleware and route registration
| | | | | } 📁 index.ts - Server entry point
| | | | | } 📁 tests
| | | | | } 📁 databaseOperations - Tests for database query services
| | | | | } 📁 getActiveRouteForPlayerSession.test.ts
| | | | | } 📁 getChildNumAndAdultDateToForever.test.ts
| | | | | } 📁 getChildrenAndAdultBySession.test.ts
| | | | | } 📁 getOrCreatePlayerSession.test.ts
| | | | | } 📁 hasSession.test.ts
| | | | | } 📁 integration
| | | | | } 📁 routes.test.ts - End-to-end route request tests
| | | | | } 📁 unit
| | | | | } 📁 campbell - Unit tests for all services and utilities
| | | | | } 📁 CheckSession.service.test.ts
| | | | | } 📁 api.jwt-guard.test.ts
| | | | | } 📁 api.test.ts
| | | | | } 📁 api.utilities.test.ts
| | | | | } 📁 game.service.test.ts
| | | | | } 📁 getNextRouteForPlayerSession.service.test.ts
| | | | | } 📁 index.test.ts
| | | | | } 📁 prisma.test.ts
| | | | | } 📁 reshape.utilities.test.ts
| | | | | } 📁 route.service.test.ts
| | | | | } 📁 middleware.test.ts
| | | | | } 📁 tsconfig.json - TypeScript config scoped to test files
| | | | | } 📁 .env - Backend environment variables
| | | | | } 📁 Dockerfile - Production image for MySQL
| | | | | } 📁 Dockerfile.pg - Production image for PostgreSQL
| | | | | } 📁 dockerfile.dev - Development image with tsx watch and Prisma
| | | | | } 📁 package.json - Backend dependencies
```

- | | | 📄 prisma.config.pg.ts - Prisma config pointing to PostgreSQL schema
- | | | 📄 prisma.config.ts - Prisma config pointing to MySQL schema
- | | | 📄 tsconfig.json - TypeScript config for backend source
- | | | 📄 vitest.config.ts - Vitest test runner config
- | | 📁 docs - In-repo documentation (separate git repo)
- | | | 📁 Admin
 - | | | | 📄 ADMIN_GUIDE.md - IT deployment, Docker, networking, backups, troubleshooting
- | | | 📁 Client
 - | | | | 📄 CLIENT_GUIDE.md - Event staff guide: what the game is, how to run events, portal usage
- | | | | 📄 main-game-qrcode.png - QR code linking to the main game at <http://10.200.0.5/>
- | | | 📁 Contributor
 - | | | | 📄 UPDATING_POLLINATOR_IMAGES.md - Non-technical guide for updating pollinator artwork
- | | | | 📁 Developer
 - | | | | | 📄 ACCESSIBILITY.md - localStorage schema, CSS overrides, TTS, font size slider
 - | | | | | 📄 ADDING_ROUTES.md - How to add new routes: seed file structure and sprites
 - | | | | | 📄 API_SPECIFICATION.md - All endpoints with exact request/response schemas, auth, and error codes
- | | | | | 📄 BACKEND_SERVICES.md - Service layer documentation
- | | | | | 📄 DATABASE_SCHEMA.md - 7-table schema, ER relationships, migration history
- | | | | | 📄 DOCKER_SETUP.md - All compose files, Dockerfiles, nginx, env vars, troubleshooting
- | | | | | | 📄 ENV_VARIABLES.md - All environment variables across all services
- | | | | | | 📄 HANDOFF.md - Project state, open work, gotchas, quick-start for new devs
- | | | | | | 📄 MYSQL_LICENSE_ISSUE.md - Why PostgreSQL is required for production
- | | | | | | 📄 NEXT_ROUTE_FLOW.md - Route deduplication algorithm and concurrency handling
- | | | | | | 📄 OVERVIEW.md - Start here: 3 apps, pages, endpoints, JWT flow, how to run
- | | | | | | 📄 PORTAL_FLOW.md - Portal state, logic, components, and services
- | | | | | | 📄 ROUTE_STATISTICS_SPEC.md - Feature spec for route statistics (in progress)
- | | | | | | 📄 ROUTE_STATS_API_REFERENCE.md - API reference for statistics endpoints
- | | | | | | 📄 SHARED_TYPES.md - Route/RouteNode/FactNode interfaces and spritesheet

coordinates

- | | | | 📄 TEST_DOCUMENTATION.md - How to run tests with per-test docs for all test files

files

- | | | | 📄 backend-flow.md - Backend architecture: startup, routing, handlers, services, Docker

Docker

- | | | | | 📄 frontend-flow.md - Frontend architecture: pages, services, components, state
- | | | | | 📄 README.md - Documentation index (audience-based)
- | | | | 📁 frontend - Next.js user-facing game client
- | | | | | 📁 public - Static assets served directly
 - | | | | | | 📁 fonts
 - | | | | | | | 📄 Antonio-Regular.ttf - Custom game font
 - | | | | | | 📁 images
 - | | | | | | | | 📄 CPLOGO.png - Conner Prairie logo
 - | | | | | | | | 📄 CP_TTS_Button.png - Text-to-speech toggle icon
 - | | | | | | | | 📄 debug_spritesheet.png - Marked-up sprite positions for development

reference

- | | | | | | 📄 minusarrow.svg - Font size decrease button
- | | | | | | 📄 placeholder.png - Default fallback image
- | | | | | | 📄 plusarrow.svg - Font size increase button
- | | | | | | 📄 spritesheet.png - All 85 pollinator sprites in one image


```
migrations
├── nginx
│   └── nginx.conf - Production reverse proxy: / to frontend, /api/ to backend, :3001
to portal
├── portal - Next.js staff-facing data collection portal
│   ├── public
│   │   ├── fonts
│   │   │   └── Antonio-Regular.ttf - Custom font
│   │   ├── images
│   │   │   ├── CPMLOGO.png - Conner Prairie logo
│   │   │   ├── CP_Wordmark.png - Conner Prairie wordmark
│   │   │   ├── favicon.ico
│   │   │   ├── favicon.ico
│   │   │   ├── minusarrow.svg
│   │   │   └── plusarrow.svg
│   │   └── src
│   │       ├── app
│   │       │   ├── export
│   │       │   │   └── page.tsx - CSV export tool for survey data
│   │       │   ├── menu
│   │       │   │   └── page.tsx - Navigation hub
│   │       │   ├── route
│   │       │   │   └── page.tsx - Route completion statistics viewer
│   │       │   ├── survey
│   │       │   │   └── page.tsx - Displays all session survey responses
│   │       │   ├── components.tsx - Portal header and navigation components
│   │       │   ├── globals.css
│   │       │   ├── layout.tsx - Portal root layout
│   │       │   └── page.tsx - Portal splash/home page
│   │       └── services
│   │           ├── csvExportService.ts - Formats survey data and triggers CSV download
│   │           ├── inputValidator.ts - Validates date ranges and filter inputs
│   │           └── surveyDatafetchService.ts - Fetches survey data from the backend API
│   ├── testing
│   │   ├── UAT
│   │   │   └── ExportSearchResults.test.tsx - User acceptance test for the export tool
│   │   ├── exportTool.test.tsx
│   │   ├── inputValidator.test.tsx
│   │   ├── layout.test.tsx
│   │   ├── portalMenu.test.tsx
│   │   ├── setup.ts - Vitest setup: mocks and globals
│   │   ├── splashPage.test.tsx
│   │   ├── surveyPortalFunction.test.tsx
│   │   └── surveyPortalStructure.test.tsx
│   ├── Dockerfile - Production image
│   ├── dockerfile.dev - Development image
│   ├── next.config.ts - Next.js config
│   ├── package.json - Portal dependencies
│   ├── tsconfig.json - TypeScript config
│   └── vitest.config.ts - Vitest test runner config
└── shared - Internal TypeScript types shared across all workspaces
    ├── index.js - Package entry point
```

```
| | 📄 package.json
| | 📄 tsconfig.json
| | 📄 types.js - Compiled JavaScript output of types.ts
| | 📄 types.ts - Route, RouteNode, and FactNode TypeScript interfaces
| | 📄 .dockerignore - Files excluded from Docker build context
| | 📄 .env - Root environment variables (committed to repo)
| | 📄 .env.pg.example - PostgreSQL environment variables (committed to repo)
| | 📄 docker-compose.dev.pg.yml - Development build with PostgreSQL
| | 📄 docker-compose.dev.yml - Development build with MySQL
| | 📄 docker-compose.prod.pg.yml - Production build with PostgreSQL
| | 📄 docker-compose.yml - Production build with MySQL
| | 📄 package.json - npm workspace root (backend, frontend, portal, shared)
| | 📄 tsconfig.base.json - Base TypeScript config shared across all workspaces
```